



Domain-Specific LLM Knowledge Injection

Maj Bregar, Jan Tršan, and Edita Džubur

Abstract

This project develops a domain-specific chatbot for Slovenian investment and tax law. Since general-purpose LLMs lack reliable domain knowledge, the proposed approach uses retrieval-augmented generation (RAG) to produce accurate, up-to-date, and verifiable responses based on external legal documents. Prompt engineering is applied to enforce structure and introduce guardrails. The system is built on a curated dataset combining legal corpora and official Slovenian sources.

Keywords

LLM, retrieval-augmented generation, chatbot, prompt engineering

Advisors: Slavko Žitnik

Introduction

Large language models (LLMs) often lack sufficiently deep domain-specific understanding to provide reliable advice in specialized fields. In domain-specific applications responses must be both accurate and verifiable, enabling users to fact-check the information through clearly provided sources. Without such guarantees, the risks of misleading or unverifiable outputs limit the usability of LLM-based systems in high-stakes domains.

This project focuses on the development of a chatbot designed to provide guidance on Slovenian investment and tax law. The domain is highly specialized and subject to regulatory detail, making it challenging for general-purpose language models to deliver consistently accurate and up-to-date responses. As a result, additional mechanisms for injecting domain-specific knowledge are required.

This report presents the design, implementation, and evaluation of a pipeline for enhancing a general-purpose LLM with domain-specific knowledge. The goal is to enable the chatbot to produce accurate, grounded, and transparent responses suitable for advisory use.

Literature Overview

Several prominent techniques exist for injecting domain-specific knowledge into large language models, differing in how knowledge is incorporated and maintained, as well as in their suitability for high-stakes domains such as law and finance.

A straightforward and widely adopted approach is prompt engineering [1], where model behavior is guided through care-

fully designed instructions, constraints, or examples. Rather than modifying the model itself, knowledge is implicitly introduced at inference time through the prompt. This makes the approach highly flexible and inexpensive, and particularly useful for rapid prototyping, enforcing output structure, or guiding reasoning processes in chatbot systems. However, its reliance on the context window limits how much domain knowledge can be injected, and performance can vary significantly depending on prompt formulation. As a result, prompt engineering alone is often insufficient in domains where correctness and reliability are critical.

In contrast, fine-tuning [2] embeds domain knowledge directly into the model's parameters by training on domain-specific data. This allows the model to internalize terminology, patterns, and expected behaviors, leading to more consistent and specialized outputs. Such properties are beneficial for tasks like document classification or maintaining a specific response style in a chatbot. At the same time, this approach comes with notable drawbacks: it is resource-intensive, depends on the availability of high-quality labeled data, and produces a static representation of knowledge that cannot easily adapt to updates, such as changes in regulations or financial markets. Furthermore, the lack of explicit source attribution limits its applicability in settings where transparency is required.

Retrieval-augmented generation (RAG) [3] addresses many of these limitations by combining pretrained model knowledge with external information sources. Instead of relying solely on internal parameters, the model retrieves relevant documents at inference time and conditions its responses on them.

This enables access to up-to-date and domain-specific information, reduces hallucinations when retrieval is effective, and supports the inclusion of explicit references. Consequently, RAG is particularly well-suited for legal and financial chatbot systems, where answers must be both accurate and verifiable. The trade-off lies in increased system complexity, as it requires a dedicated retrieval pipeline, and in sensitivity to retrieval quality, which directly impacts response reliability and latency.

In practice, these methods are rarely used in isolation. Prompt engineering is often employed to structure and constrain outputs, fine-tuning to ensure consistent behavior, and RAG to provide grounded, up-to-date knowledge, resulting in hybrid systems that balance flexibility, accuracy, and transparency.

Proposed Dataset and Data Sources

In this project, we focus on developing a domain-specific AI assistant for the Slovenian legal domain, with a particular emphasis on tax and investment legislation (e.g., *ZDoh-2*, *ZDavP-2*). To achieve this, we construct a comprehensive dataset by combining an existing legal corpora with additional publicly available Slovenian legal and linguistic resources.

Core Legal Corpus

The primary dataset is based on the Slovenian legal corpus provided within the LLM4DH project [4]. This corpus contains a large collection of Slovenian legislation, which serves as the main source of truth for the system. It includes structured legal documents such as laws, regulations, and amendments, making it suitable for tasks such as retrieval-augmented generation (RAG), legal question answering, and document grounding.

Supplementary Legal Data Sources

To enhance the coverage and practical applicability of the system, we incorporate additional Slovenian legal resources:

- **PISRS (Pravno-informacijski sistem RS)**¹: A centralized repository of Slovenian legislation, providing structured access to laws, including tax-related acts such as *ZDoh-2*. This source enables article-level segmentation and metadata extraction.
- **Uradni list RS**²: The official gazette of the Republic of Slovenia, containing original publications of laws and amendments. This source is useful for validating legal texts and tracking legislative changes.
- **FURS (Financial Administration of RS)**³: Provides tax-related guidelines, explanations, and frequently asked questions. This resource is particularly relevant for constructing question-answer datasets aligned with real-world tax queries.

¹<https://pisrs.si>

²<https://www.uradni-list.si>

³<https://www.fu.gov.si>

- **e-Uprava**⁴: A government portal offering simplified explanations of legal procedures and regulations. This source is useful for bridging the gap between formal legal language and user-friendly descriptions.

CLARIN.SI Legal and Linguistic Corpora

In addition to legal texts, we incorporate corpora from the CLARIN.SI infrastructure to support linguistic processing and model development.

- **Slovenian Legal Corpus (CLARIN.SI)**⁵: This corpus contains Slovenian legal texts annotated with linguistic information and metadata. It is particularly useful for training and evaluating NLP models in the legal domain, as well as for tasks such as named entity recognition and terminology extraction.
- **siParl Parliamentary Corpus**⁶: A large corpus of Slovenian parliamentary debates covering the period from 1990 to 2018, containing over 200 million words and rich metadata about speakers and sessions. This dataset provides contextual information about legislative discussions and policy development [5].

CLARIN resources are typically provided in structured formats such as XML or CONLL-U and include linguistic annotations (tokenization, lemmatization, part-of-speech tagging), which facilitates downstream NLP tasks and model fine-tuning.

Dataset Construction Strategy

The final dataset will be constructed by combining the above sources into a unified corpus. The main steps include:

- **Data cleaning and normalization**: Converting documents from HTML/PDF/XML into a consistent textual format.
- **Segmentation**: Splitting legal documents into meaningful units (e.g., articles, paragraphs).
- **Metadata extraction**: Capturing information such as law title, article number, and publication date.
- **Domain filtering**: Selecting tax- and investment-related legislation (e.g., *ZDoh-2*).
- **Alignment with explanatory sources**: Linking legal provisions with interpretations from FURS and e-Uprava.

Overall, the combination of authoritative legal texts, explanatory government resources, and linguistically annotated corpora provides a strong foundation for building a reliable and domain-specific legal AI assistant.

⁴<https://e-uprava.gov.si>

⁵<https://www.clarin.si/repository/xmlui/handle/11356/2095>

⁶<https://www.clarin.si/repository/xmlui/handle/11356/1300>

Methodology

Chatbot systems designed for domains such as investing and tax law require a high level of accuracy, reliability, and transparency in their responses. To address these requirements, the proposed approach primarily relies on retrieval-augmented generation (RAG). By integrating external domain-specific documents into the generation process, the system can provide up-to-date and factually grounded answers. Importantly, retrieved documents can be cited as references, allowing users to verify the information, which is essential in high-stakes decision-making contexts.

0.1 Chunking Strategy

The retrieval component of the chatbot is built by converting legal documents into structured text chunks and embedding them into a vector index. The input consists of COLESLAW-style JSONL records, where each record contains metadata such as the law title, identifier, and full legal text.

The chunking process first filters the input records according to one of three strategies. In the default strategy, records are selected using a combination of explicitly whitelisted laws and keyword-based relevance filtering for tax and investment terminology. In the whitelist strategy, only a predefined set of core laws is retained, including *ZDoh-2*, *ZDavP-2*, *ZDDPO-2*, *ZDDV-1*, *ZTFI-1*, *ZISDU-3*, *ZBan-3*, and *ZGD-1*. In the all-laws strategy, filtering is disabled and all input records are processed; records that cannot be parsed into articles may optionally be preserved as fallback document-level chunks.

Each selected legal document is then segmented according to its internal legal structure. The parser detects article markers such as *1. člen*, extracts article headings when present, assigns chapter or section metadata, and splits article bodies into paragraph-level units such as *(1)*, *(2)*, and similar legal subdivisions. For long provisions, the parser applies additional splitting based on legal list boundaries, including numbered and lettered items. This avoids arbitrary cuts in the middle of words or legal clauses and helps preserve the semantic integrity of each chunk.

Each generated chunk contains both the raw chunk text and structured metadata. The metadata includes the law title, law code, article number, article heading, paragraph or item number, chapter, document identifiers, and domain tags such as *tax*, *income_tax*, *capital_gains*, *investment*, or *vat*. A formatted version of the chunk is also created, combining the legal text with its metadata. This formatted representation is used for embedding so that retrieval can benefit from both the content of the provision and its legal context.

This strategy is designed to balance retrieval precision with legal interpretability. Article and paragraph-level chunks are small enough to support focused retrieval, while the attached metadata preserves the legal source context needed for citation and verification.

0.2 Chunk Embedding

Chunk embeddings are generated using the multilingual sentence-transformer model *BAAI/bge-m3* [6], which provides strong cross-lingual performance and works well for morphologically rich languages such as Slovenian.

The resulting vectors are normalized by default and stored as a NumPy matrix. A FAISS index is then optionally constructed using inner product search over the normalized embeddings, which corresponds to cosine similarity. The final output directory contains the chunk file, embedding matrix, FAISS index.

0.3 RAG Pipeline

Upon receiving a user query, the text is embedded using *BAAI/bge-m3*, and the top 50 most similar chunks are retrieved from the RAG chunk store. These candidates are then reranked using *BAAI/bge-reranker-large* [7], with the top 10 most relevant chunks retained. The selected chunks, together with their metadata (e.g., law name, article, paragraph), are used to construct a structured RAG prompt that clearly delineates the legal context and provides explicit instructions for grounded answering.

The RAG prompt, along with the user query, system-level formatting instructions, and conversation history, is sent to a vLLM server running *Llama-3.1-8B-Instruct* [8] via the OpenAI API message interface. The prompt enforces strict reliance on the provided legal context, requiring the model to cite relevant laws and articles, avoid hallucinations, and explicitly indicate when the answer cannot be determined from the retrieved sources.

0.4 Future Pipeline Extensions

In addition to RAG, prompt engineering will be employed to guide the behavior of the language model. Carefully designed prompts will define the expected response structure, enforce clarity and precision, and introduce guardrails to reduce ambiguous or misleading outputs. This combination ensures that responses are not only grounded in reliable sources but also presented in a consistent and controlled manner.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [2] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al.

Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

- [4] LLM4DH Project. Llm4dh slovenian legal corpus, 2026. Internal dataset provided within the LLM4DH project.
- [5] CLARIN.SI. siparl: Slovene parliamentary corpus 1990–2018, 2019. Annotated corpus of Slovenian parliamentary proceedings.
- [6] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 4(5), 2024.
- [7] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muenighoff. C-pack: Packaged resources to advance general chinese embedding, 2023.
- [8] Meta Llama. Meta llama-3.1-8b-instruct, 2024.